

Laboratorio 1: Tiny MC

Computación Paralela - FAMAF

Javier Cremona
Marcelo Castillo

April 15, 2021

Sistema Operativo: Ubuntu 20.04.2 LTS

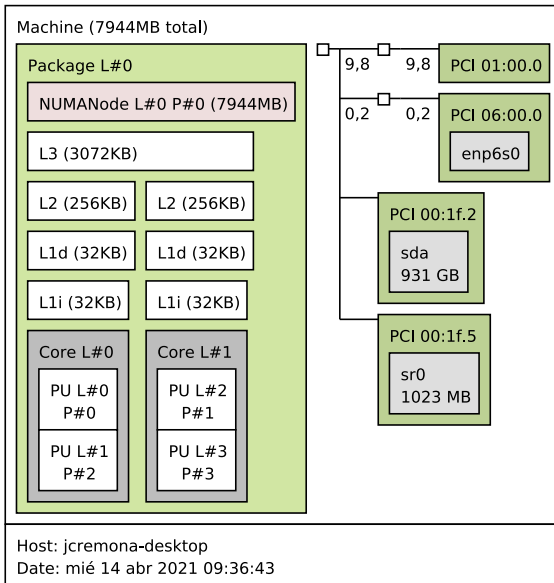
Kernel: Linux 5.8.0-48-generic

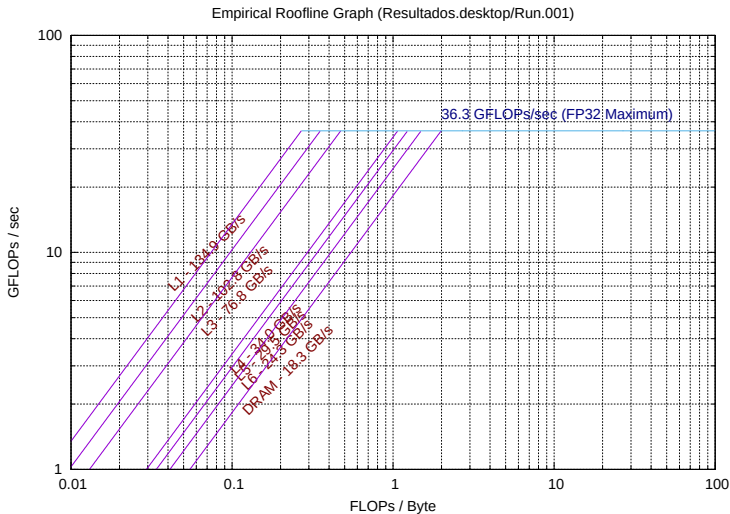
Arquitectura: x86_64

CPU: Intel Core i3-2100 CPU @ 3.10GHz - (Sandy Bridge)

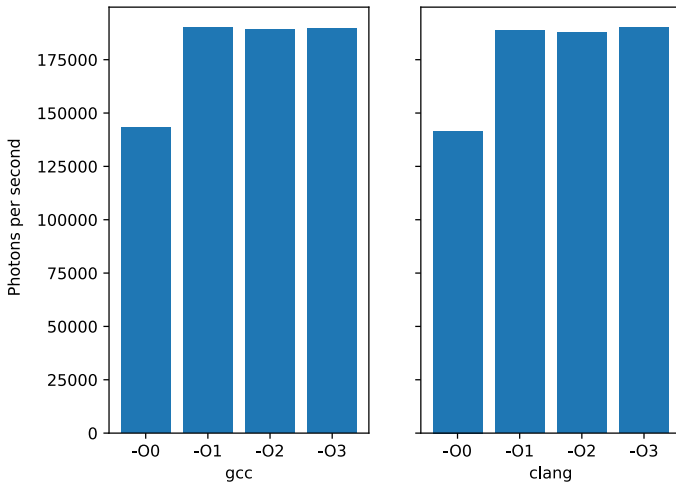
RAM: tamaño 8GB, velocidad 1333 MT/s

- ranura: A0, tamaño: 4GiB, anchura: 2244 bits
- ranura: A2, tamaño: 4GiB, anchura: 2244 bits

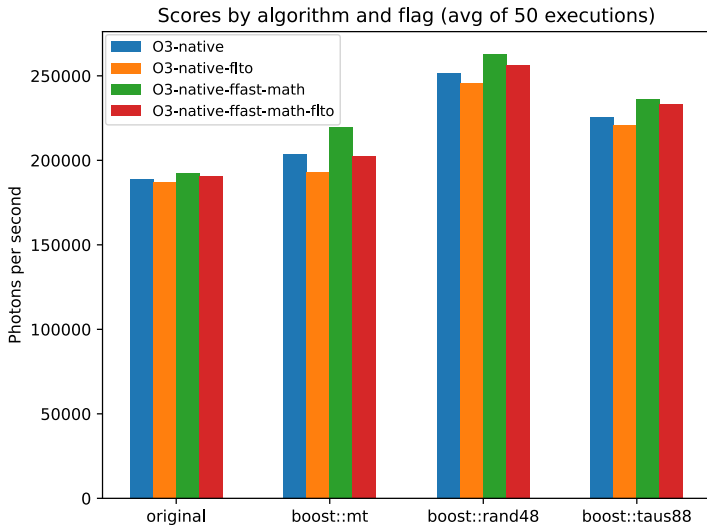




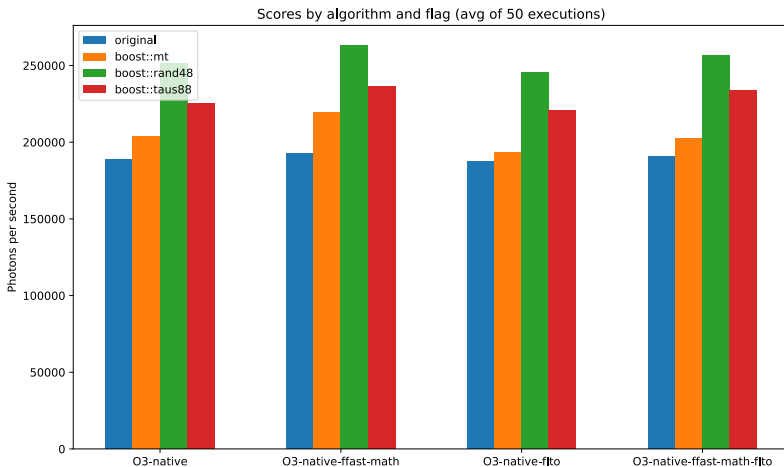
Average (50 samples) photons per sec by compiler and optimization flags
(in addition to march=native).



Eligiendo un nuevo PNRG



Eligiendo un nuevo PNRG



Cambios en el código

Hasta el momento, cambiando `rand()` por `boost::rand48` y agregando `-O3`, `-march=native` y `-ffast-math`, obtuvimos un score de 261K photons per second (pps), habiendo partido un código original que promediaba 140K pps (sin banderas de optimización, con `O0`).

Se proponen distintos cambios en el código.

Cambios en el código: usar un array en lugar de dos

```
for (;;) {  
    float t = -logf(rand() / (float)RAND_MAX); /* move */  
    x += t * u;  
    y += t * v;  
    z += t * w;  
  
    unsigned int shell = sqrtf(x * x + y * y + z * z) * shells_per_mfp; /* absorb */  
    if (shell > SHELLS - 1) {  
        shell = SHELLS - 1;  
    }  
    heat[shell] += (1.0f - albedo) * weight;  
    heat2[shell] += (1.0f - albedo) * (1.0f - albedo) * weight * weight; /* add up squares */  
    weight *= albedo;  
}
```

Figure: Loop dentro del método photon()

Cambios en el código: usar un array en lugar de dos

Figure: Código original (izq.) y la modificación aplicada (der.).

```
// global state,  
// heat and heat square  
// in each shell  
static float heat[SHELLS];  
static float heat2[SHELLS];
```

```
// global state  
struct heat_struct {  
    float heat; // heat  
    float heat2; // heat square  
};  
  
static heat_struct heat_array[SHELLS];
```

Cambios en el código: mover el break

Figure: Código original (izq.) y la modificación aplicada (der.).

```
/* New direction, rejection method */
float xi1, xi2;
do {
    xi1 = 2.0f * rand() / (float)RAND_MAX - 1.0f;
    xi2 = 2.0f * rand() / (float)RAND_MAX - 1.0f;
    t = xi1 * xi1 + xi2 * xi2;
} while (1.0f < t);
u = 2.0f * t - 1.0f;
v = xi1 * sqrtf((1.0f - u * u) / t);
w = xi2 * sqrtf((1.0f - u * u) / t);

if (weight < 0.001f) { /* roulette */
    if (rand() / (float)RAND_MAX > 0.1f)
        break;
    weight /= 0.1f;
}
```

```
/* Rejection method */
if (weight < 0.001f) { /* roulette */
    if (random_float() > 0.1f)
        break;
    weight /= 0.1f;
}

/* New direction */
float xi1, xi2;
do {
    xi1 = 2.0f * random_float() - 1.0f;
    xi2 = 2.0f * random_float() - 1.0f;
    t = xi1 * xi1 + xi2 * xi2;
} while (1.0f < t);
u = 2.0f * t - 1.0f;
v = xi1 * sqrtf((1.0f - u * u) / t);
w = xi2 * sqrtf((1.0f - u * u) / t);
```

Usando ambas modificaciones se logra una mejora del 6%, llegando a 275K pps.

Un intento de cambiar la representación

Para el cálculo de la nueva dirección de un fotón, este algoritmo samplea puntos (x,y) en el cuadrado de lado 2 centrado en el origen, pero se queda con aquellos que pertenecen al círculo centrado en el origen con radio 1. Intentamos usar coordenadas polares para generar puntos dentro del círculo, evitando el do-while.

```
/* New direction */  
float xi1, xi2, ang;  
ang = (random_float()) * M_PI * 2;  
t = sqrtf(random_float());  
xi1 = t * cosf(ang);  
xi2 = t * sinf(ang);  
  
u = 2.0f * t - 1.0f;  
v = xi1 * sqrtf((1.0f - u * u) / t);  
w = xi2 * sqrtf((1.0f - u * u) / t);
```

Figure: Código con coordenadas polares.

Un intento de cambiar la representación

Pero las funciones trigonométricas resultaron muy caras!

Samples: 548 of event 'cycles', Event count (approx.): 390088878			
Overhead	Command	Shared Object	Symbol
75,57%	tiny_mc	tiny_mc	[.] main
12,91%	tiny_mc	libm-2.31.so	[.] __sincosf_sse2
8,61%	tiny_mc	libm-2.31.so	[.] __logf_sse2
0,78%	tiny_mc	tiny_mc	[.] 0x00000000000011d4
0,39%	tiny_mc	tiny_mc	[.] 0x0000000000001124
0,29%	tiny_mc	tiny_mc	[.] 0x0000000000001120

Figure: Salida de perf para el programa que usa coordenadas polares.

Una posible solución sería usar aproximaciones computacionalmente eficientes para seno y coseno. El precio a pagar sería la precisión, algunas aproximaciones sólo son precisas para un subconjunto pequeño del dominio de seno y coseno. Decidimos dejar la versión original.

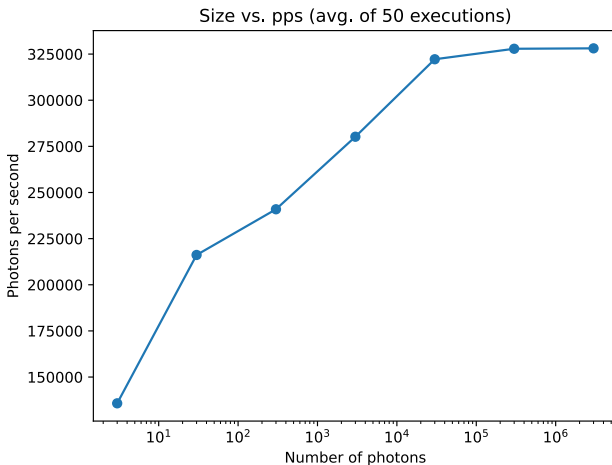
Finalmente, usamos FDO con las flags `-fgenerate-profile`, y `-fgenerate-use`.

Para 50 corridas, en promedio obtuvimos **323K pps**.

- Las iteraciones del for principal del método photons dependen de los valores de la iteración anterior, pues la nueva posición del fotón se calcula a partir de la posición anterior.
- La bandera -funroll-loops no nos mejora nada.
- El compilador avisa que no puede vectorizar ningún bucle del programa (-fopt-info-vec-missed).
- Loop blocking no nos sirve, ya que saltamos a lugares aleatorios del array.
- Y loop peeling?

Tamaño del problema

El programa usa un tamaño (casi) constante de memoria, un array con aproximadamente 200 floats.



- Cambiamos el PRNG.
- -flt no nos aportó mejoras.
- Buscamos evitar la generación de números aleatorios que no se van a usar.
- $\sin$, $\cos$ y $\log$ nos resultan caras.
- FDO mejora el programa en casi 20%.
- Loop unrolling/blocking no nos mejora el algoritmo.
- Pasamos de 140K pps a 323K pps.
- Por mejorar: puntos (x,y) sampleados en el círculo de radio 1 y $\log$.